LARS JANSEN

Senior Software Engineer/ DevOps Engineer (Cloud-Native Platform)

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior DevOps/SRE engineer building **cloud-native** platforms for regulated enterprise programs across telecom and multi-industry clients, shipping **Kubernetes** workloads, **GitLab CI** delivery, and **Terraform/Terragrunt** IaC that scales safely across environments. I automate golden paths with **Go 1.18–1.22** and **Python 3.9–3.12**, harden runtime security (RBAC/PSA, network policies, image signing), and drive observability with **OpenTelemetry 1.x**, Prometheus, Grafana, and Datadog. Known for incident leadership, cost control, and clean module design. Strong in migrations like **Kubernetes 1.21–1.29** and Terraform state refactors. at scale.

SKILLS

- **Infrastructure-as-Code** — **Terraform 1.3–1.7**, **Terragrunt 0.38–0.58**, reusable modules, remote state, policy-as-code patterns, drift detection, state moves/imports, multi-env promotion, cost tagging and chargeback-ready design.
- **CI/CD & Release Engineering** — **GitLab CI**, monorepo + multi-repo pipelines, build caching, parallelized test stages, security scanning gates (SAST/DAST/dependency), artifact signing, blue/green + canary rollouts, versioned Helm releases, ChatOps approvals.
- **Containers & Kubernetes** — **Kubernetes 1.21–1.29**, **Helm 3.6–3.14**, **Docker 20.10–26**, deployments/statefulsets/daemonsets/jobs/cronjobs, sidecars/init containers, HPA/VPA, PDBs, ingress, service mesh-friendly patterns, Kubernetes API object model troubleshooting.
- **Cloud Platforms** — **AWS** (EKS, IAM, VPC, ALB/NLB, Route 53, S3, RDS, CloudWatch), **Azure** (AKS, ACR, Key Vault, Monitor), **GCP** (GKE, IAM, Cloud Storage) with enterprise networking and least-privilege IAM.
- **Observability & Reliability** — **OpenTelemetry 1.x**, Prometheus, Grafana, Datadog, ELK/EFK patterns, SLOs/SLIs, alert tuning, runbooks, on-call hygiene, incident response, capacity planning, performance profiling for runtime + nodes.
- **Security & Compliance** — RBAC, Pod Security Admission, NetworkPolicies, secret management, image scanning/signing, audit logging, TLS/mTLS patterns, hardening baselines, change control evidence, "Deutsche Telekom-grade" controls alignment.
- **Automation & Scripting** — **Python 3.9–3.12**, **Go 1.18–1.22**, internal CLI tools, GitOps helpers, cluster validation scripts, pipeline generators, API-based integrations (Kubernetes API, GitLab API).

WORK HISTORY

Senior Software Engineer
RavenTrack

August 2021 to November 2025

- I designed a **Terraform/Terragrunt** module library that standardized VPC, EKS, and IAM patterns, cutting environment setup from days to hours with consistent guardrails.
- I rebuilt **GitLab CI** pipelines into reusable templates with matrix builds, caching, and release gates, reducing flaky deployments by tightening artifact promotion and rollback rules.
- I migrated multiple services from ad-hoc manifests to **Helm** charts with strict values schemas, enabling predictable multi-environment releases and safer config changes.
- I resolved a recurring production outage caused by mis-sized node pools and noisy neighbors by adding HPA/VPA tuning, resource requests/limits, and topology spread constraints.
- I debugged a hard-to-trace networking issue where intermittent 502s were triggered by conntrack exhaustion, fixing it via node kernel tuning, keepalive settings, and scaled NAT capacity.
- I implemented security hardening with **RBAC**, Pod Security Admission, and NetworkPolicies, closing privilege gaps discovered during internal audits without blocking delivery.
- I introduced **OpenTelemetry** tracing across critical services, turning "unknown latency" incidents into actionable spans and reducing MTTR with consistent correlation IDs.
- I built automated image scanning and dependency checks into the pipeline, blocking vulnerable releases while preserving developer speed through clear remediation feedback.
- I led a **Kubernetes 1.21–1.29** upgrade path using canaries and compatibility testing, preventing API breakages by pre-validating CRDs and controller versions.
- I created a GitOps-style promotion flow that separated build from deploy, ensuring the same artifact moved dev → staging → prod with an auditable change trail.
- I fixed a data-loss risk in batch jobs where retries reprocessed messages, adding idempotency keys, dead-letter handling, and bounded backoff to protect downstream systems.
- I optimized Dockerfiles and runtime startup time by removing heavy layers, pinning base images, and tuning JVM/Node flags where needed to cut cold-start latency.
- I implemented HA strategies for ingress and critical stateful workloads, combining PDBs, multi-AZ node groups, and controlled rollouts to avoid brownouts.
- I partnered with developers on "cloud-native by default" design reviews, focusing on graceful shutdown, readiness probes, and safe retries to improve reliability.
- I drove cost controls by enforcing tagging, rightsizing nodes, and cleaning orphaned resources, producing measurable savings without compromising availability.

Software Engineer
Infinite Loop
June 2018 to July 2021

- I built CI/CD workflows that packaged services into Docker images, ran unit/integration tests, and deployed to Kubernetes with environment-specific policies and approvals.
- I fixed a pipeline-breaking bug where nondeterministic builds caused "works in CI, fails in prod," solving it with locked dependencies, reproducible builds, and artifact immutability.
- I automated infrastructure provisioning using Terraform modules and remote state, removing manual console steps and making environments consistent across teams.
- I improved release safety by implementing blue/green deployment strategies with health checks and automated rollback on error-rate spikes.
- I handled production incidents triggered by misconfigured ingress timeouts, diagnosing it from logs/metrics and correcting annotations plus upstream keepalive behavior.
- I introduced cluster-level monitoring with Prometheus and Grafana dashboards, then tuned alerts to reduce noise while catching real saturation early.
- I wrote Python tooling to validate Kubernetes manifests and Helm values before merge, preventing runtime failures from schema drift and bad defaults.
- I migrated legacy cron workloads into Kubernetes CronJobs with concurrency policies and resource limits, eliminating overlapping runs that caused database lock storms.
- I enforced least-privilege access by restructuring RBAC roles per namespace, reducing "cluster-admin everywhere" risk while keeping developer workflows smooth.
- I improved container runtime performance by optimizing Docker layers, reducing image size, and adding health endpoints for faster autoscaling decisions.
- I partnered with product teams to standardize Git workflows and release branching, improving traceability and reducing hotfix conflicts in critical releases.

Software Developer
Datamart
October 2015 to May 2018

- I supported early container adoption by containerizing internal services and creating repeatable build scripts that eliminated environment drift and onboarding friction.
- I fixed a recurring deployment failure caused by missing secrets and inconsistent config, solving it with environment-driven settings and centralized secret injection.
- I automated nightly batch pipelines and backfills, adding retry logic and safe checkpoints so partial failures did not corrupt downstream reporting tables.
- I improved database reliability by introducing safe migrations, indexing hotspots, and query profiling to prevent lock contention during peak reporting windows.
- I resolved a production bug where log rotation filled disks and crashed services, implementing monitoring plus cleanup automation with clear on-call runbooks.
- I introduced structured logging and correlation IDs, making it possible to trace failures across services and reduce investigation time during incidents.
- I helped migrate legacy scripts into Python automation with consistent error handling, exit codes, and alert hooks for operational visibility.
- I reduced "snowflake server" risk by documenting baseline configs and standardizing provisioning steps, enabling faster recovery during hardware failures.
- I partnered with developers to implement basic release checklists and smoke tests, catching config and dependency issues before they reached users.

EDUCATION

Bachelor's degree in Computer Science
University of Amsterdam
2011 to 2015

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Enterprise Kubernetes Landing Zone (Multi-Environment Platform)

- I delivered a standardized **Terraform/Terragrunt** foundation for networks, IAM, and managed Kubernetes, plus **Helm**-based app packaging that enabled consistent deployments across multiple environments with audit-friendly change history.

GitLab CI Delivery Platform (Reusable Pipelines & Security Gates)

- I created shared **GitLab CI** templates with caching, parallel test stages, artifact promotion, and security scanning gates, reducing deployment risk while keeping release velocity high.